

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I VENNA MEENAKSHI REDDY(192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Venna Meenakshi Reddy

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results

Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Operations: Insert, Delete, Sequential Scan.

```
class HeapFile:
    def __init__(self):
        self.records = []

    def insert(self, record):
        self.records.append(record)
        print("Inserted:", record)

    def delete(self, key):
        for i, record in enumerate(self.records):
            if record["id"] == key:
                deleted = self.records.pop(i)
                print("Deleted:", deleted)
                return
        print("Record not found")

    def sequential_scan(self):
        print("\nSequential Scan:")
        if not self.records:
            print("Heap file is empty")
            return

        for record in self.records:
            print(record)

# Test
heap = HeapFile()
```

```
heap.insert({"id": 101, "name": "Arun"})
heap.insert({"id": 102, "name": "Bala"})
heap.insert({"id": 103, "name": "Charan"})
heap.insert({"id": 104, "name": "Divya"})
```

```
heap.sequential_scan()
```

```
heap.delete(102)
```

```
heap.sequential_scan()
```

Expected Output

```
Inserted: {'id': 101, 'name': 'Arun'}
Inserted: {'id': 102, 'name': 'Bala'}
Inserted: {'id': 103, 'name': 'Charan'}
Inserted: {'id': 104, 'name': 'Divya'}
```

Sequential Scan:

```
{'id': 101, 'name': 'Arun'}
{'id': 102, 'name': 'Bala'}
{'id': 103, 'name': 'Charan'}
{'id': 104, 'name': 'Divya'}
```

Deleted: {'id': 102, 'name': 'Bala'}

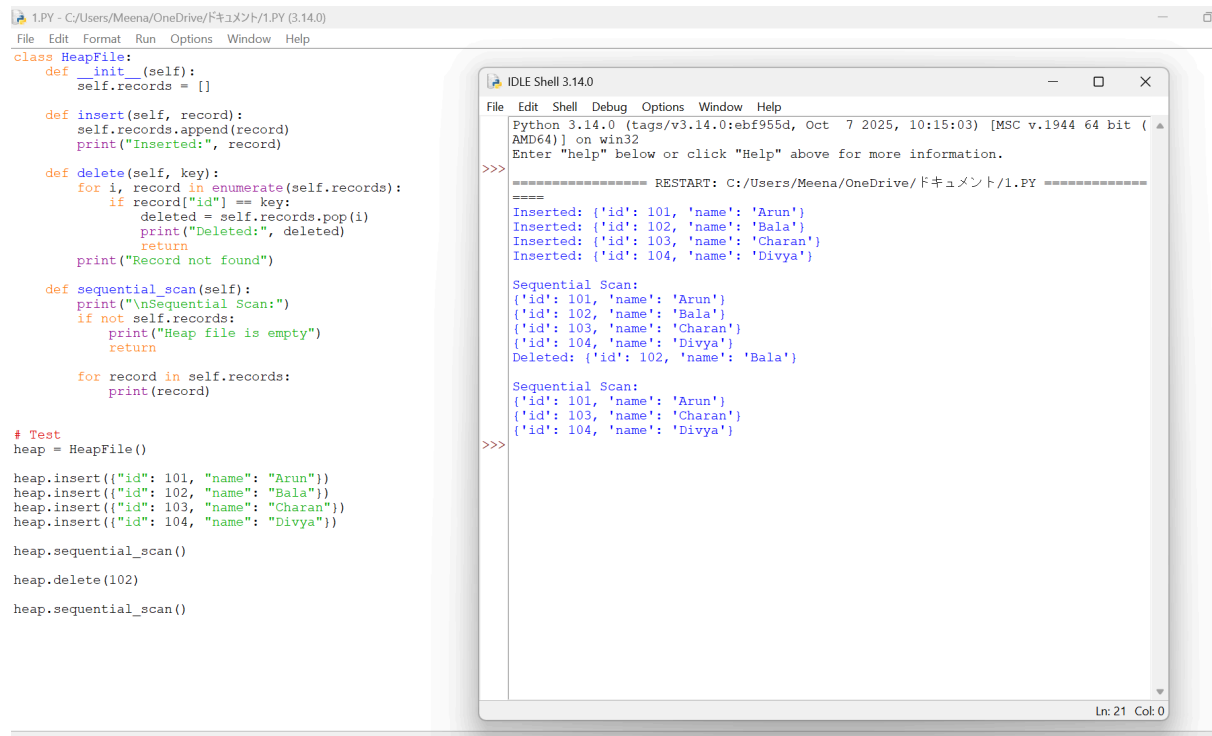
Sequential Scan:

```
{'id': 101, 'name': 'Arun'}
{'id': 103, 'name': 'Charan'}
{'id': 104, 'name': 'Divya'}
```

Complexity:

- Insert: $O(1)$
- Delete: $O(n)$
- Sequential scan: $O(n)$

OUTPUT:



The image shows a Python IDE with two windows. The left window displays the source code for a `HeapFile` class. The right window, titled 'IDLE Shell 3.14.0', shows the execution output. The code defines methods for inserting, deleting, and scanning records. The test section creates a `HeapFile` object and performs several operations: inserting four records, scanning sequentially, deleting a record, and scanning again. The output in the shell window shows the state of the records after each operation.

```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help

class HeapFile:
    def __init__(self):
        self.records = []

    def insert(self, record):
        self.records.append(record)
        print("Inserted:", record)

    def delete(self, key):
        for i, record in enumerate(self.records):
            if record["id"] == key:
                deleted = self.records.pop(i)
                print("Deleted:", deleted)
                return
        print("Record not found")

    def sequential_scan(self):
        print("\nSequential Scan:")
        if not self.records:
            print("Heap file is empty")
            return
        for record in self.records:
            print(record)

# Test
heap = HeapFile()

heap.insert({"id": 101, "name": "Arun"})
heap.insert({"id": 102, "name": "Bala"})
heap.insert({"id": 103, "name": "Charan"})
heap.insert({"id": 104, "name": "Divya"})

heap.sequential_scan()

heap.delete(102)

heap.sequential_scan()
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
Inserted: {'id': 101, 'name': 'Arun'}
Inserted: {'id': 102, 'name': 'Bala'}
Inserted: {'id': 103, 'name': 'Charan'}
Inserted: {'id': 104, 'name': 'Divya'}

Sequential Scan:
{'id': 101, 'name': 'Arun'}
{'id': 102, 'name': 'Bala'}
{'id': 103, 'name': 'Charan'}
{'id': 104, 'name': 'Divya'}
Deleted: {'id': 102, 'name': 'Bala'}

Sequential Scan:
{'id': 101, 'name': 'Arun'}
{'id': 103, 'name': 'Charan'}
{'id': 104, 'name': 'Divya'}

>>>
```

Ln: 21 Col: 0

2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

For 500 student records, the hash table uses chaining to handle collisions.

TABLE_SIZE = 503

```
class StaticHash:
    def __init__(self):
        self.table = [[] for _ in range(TABLE_SIZE)]

    def hash_function(self, student_id):
        return student_id % TABLE_SIZE

    def insert(self, student_id, name):
        index = self.hash_function(student_id)

        for record in self.table[index]:
            if record[0] == student_id:
                print("Duplicate Student ID:", student_id)
                return

        self.table[index].append((student_id, name))

    def search(self, student_id):
        index = self.hash_function(student_id)

        for record in self.table[index]:
            if record[0] == student_id:
                return record

        return None
```

```

def display_bucket(self, student_id):
    index = self.hash_function(student_id)
    print("Bucket", index, ":", self.table[index])

# Test with 500 records
hash_table = StaticHash()

for i in range(1, 501):
    hash_table.insert(1000 + i, "Student" + str(i))

# Collision demonstration
hash_table.insert(1503, "CollisionStudent")

hash_table.display_bucket(1001)
hash_table.display_bucket(1503)

print("Search Result:", hash_table.search(1250))

```

Key Features

- Table size: 503
- Records: 500
- Hash function:

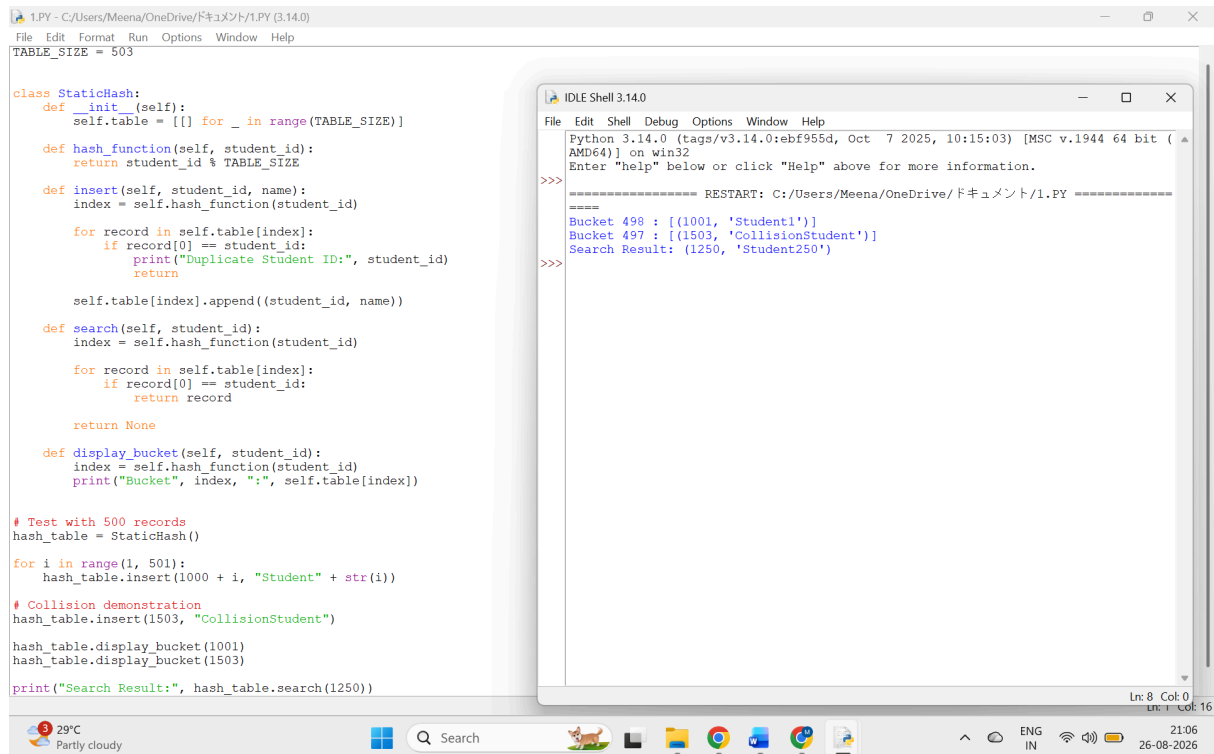
$h(\text{key}) = \text{key} \text{ MOD } 503$

- Collision handling: Chaining

Average complexity:

- Insert: $O(1)$
- Search: $O(1)$
- Worst case: $O(n)$

OUTPUT:



The screenshot displays a Python IDE with a file named '1.PY' and an associated IDLE Shell window. The Python code implements a hash table using separate chaining. It defines a 'StaticHash' class with methods for initialization, hashing, insertion (checking for collisions), searching, and displaying buckets. The code includes a test with 500 records, a collision demonstration at index 1503, and a search for 'Student250'.

```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help
TABLE_SIZE = 503

class StaticHash:
    def __init__(self):
        self.table = [[] for _ in range(TABLE_SIZE)]

    def hash_function(self, student_id):
        return student_id % TABLE_SIZE

    def insert(self, student_id, name):
        index = self.hash_function(student_id)
        for record in self.table[index]:
            if record[0] == student_id:
                print("Duplicate Student ID:", student_id)
                return
        self.table[index].append((student_id, name))

    def search(self, student_id):
        index = self.hash_function(student_id)
        for record in self.table[index]:
            if record[0] == student_id:
                return record
        return None

    def display_bucket(self, student_id):
        index = self.hash_function(student_id)
        print("Bucket", index, ":", self.table[index])

# Test with 500 records
hash_table = StaticHash()
for i in range(1, 501):
    hash_table.insert(1000 + i, "Student" + str(i))

# Collision demonstration
hash_table.insert(1503, "CollisionStudent")

hash_table.display_bucket(1001)
hash_table.display_bucket(1503)

print("Search Result:", hash_table.search(1250))
```

The IDLE Shell window shows the execution output:

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
>>>
Bucket 498 : [(1001, 'Student1')]
Bucket 497 : [(1503, 'CollisionStudent')]
Search Result: (1250, 'Student250')
>>>
```

The status bar at the bottom indicates the current position is Line 8, Column 0, and the system clock shows 21:06 on 26-08-2026.

3. Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

```
class SortedFile:
```

```
    def __init__(self):
```

```
        self.records = []
```

```
    def insert(self, key, name):
```

```
        self.records.append((key, name))
```

```
        self.records.sort(key=lambda x: x[0])
```

```
    def binary_search(self, key):
```

```
        low = 0
```

```
        high = len(self.records) - 1
```

```
        while low <= high:
```

```
            mid = (low + high) // 2
```

```
            if self.records[mid][0] == key:
```

```
                return self.records[mid]
```

```
            elif self.records[mid][0] < key:
```

```
                low = mid + 1
```

```
            else:
```

```
                high = mid - 1
```

```
        return None
```

```
    def display(self):
```

```
        print("Sorted File:")
```

```
        for record in self.records:
```

```
            print(record)
```

```
# Test
file = SortedFile()

file.insert(104, "Divya")
file.insert(101, "Arun")
file.insert(103, "Charan")
file.insert(102, "Bala")

file.display()

key = 103
result = file.binary_search(key)

if result:
    print("\nRecord Found:", result)
else:
    print("\nRecord Not Found")
```

Output

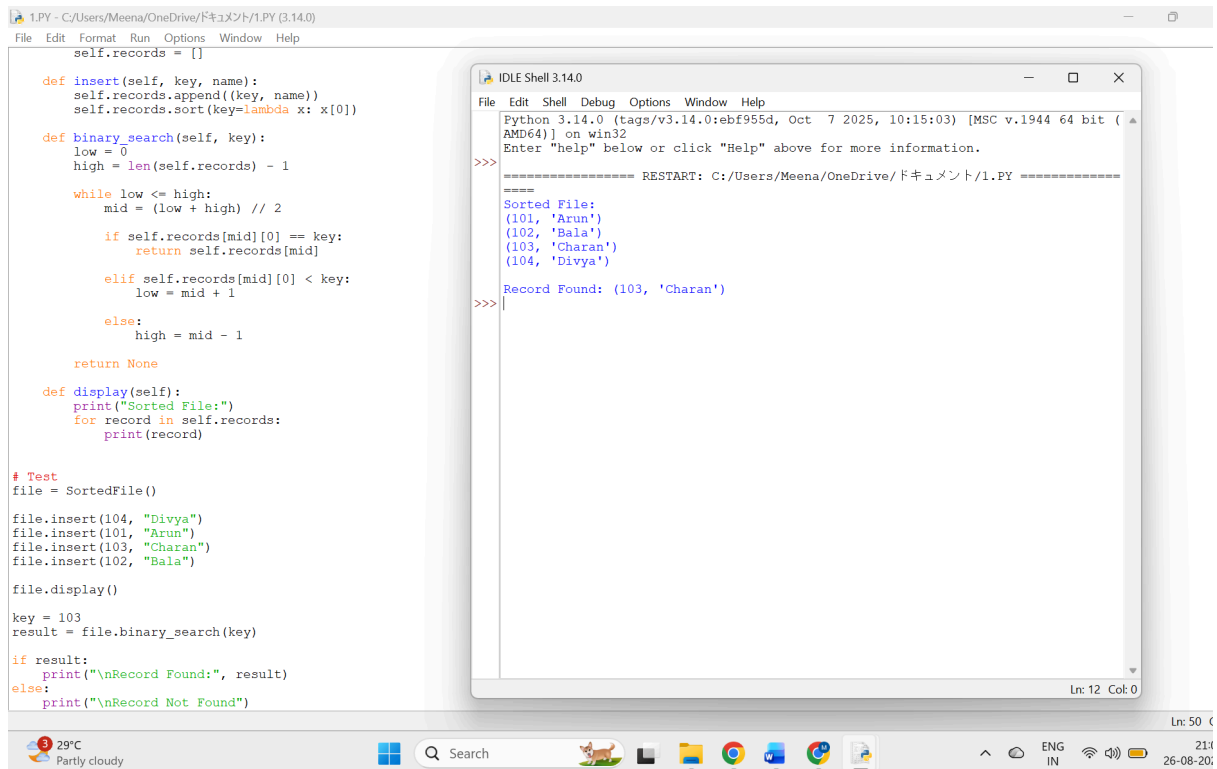
Sorted File:
(101, 'Arun')
(102, 'Bala')
(103, 'Charan')
(104, 'Divya')

Record Found: (103, 'Charan')

Complexity:

- Binary search: $O(\log n)$
- Display: $O(n)$
- Insertion with sorting: $O(n \log n)$

OUTPUT:



The screenshot displays a Python IDE with two windows. The main window shows a Python script for a binary search algorithm. The script defines a `SortedFile` class with methods `insert`, `binary_search`, and `display`. A test section creates an instance of `SortedFile`, inserts four records (104, 'Divya', 101, 'Arun', 103, 'Charan', 102, 'Bala'), and then performs a binary search for the key 103. The output window shows the sorted file and the result of the search.

```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help

self.records = []

def insert(self, key, name):
    self.records.append((key, name))
    self.records.sort(key=lambda x: x[0])

def binary_search(self, key):
    low = 0
    high = len(self.records) - 1

    while low <= high:
        mid = (low + high) // 2

        if self.records[mid][0] == key:
            return self.records[mid]

        elif self.records[mid][0] < key:
            low = mid + 1

        else:
            high = mid - 1

    return None

def display(self):
    print("Sorted File:")
    for record in self.records:
        print(record)

# Test
file = SortedFile()

file.insert(104, "Divya")
file.insert(101, "Arun")
file.insert(103, "Charan")
file.insert(102, "Bala")

file.display()

key = 103
result = file.binary_search(key)

if result:
    print("\nRecord Found:", result)
else:
    print("\nRecord Not Found")
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
Sorted File:
(101, 'Arun')
(102, 'Bala')
(103, 'Charan')
(104, 'Divya')
Record Found: (103, 'Charan')
>>>
```

Ln: 12 Col: 0

Ln: 50 Col: 0

29°C Partly cloudy

Search

ENG IN

21°C 26-08-2025

4. Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

For order 4:

- Maximum children = 4
- Maximum keys = 3

Keys:

10, 20, 5, 6, 12, 30, 7, 17

class BTreeNode:

```
def __init__(self, leaf=False):  
    self.keys = []  
    self.children = []  
    self.leaf = leaf
```

class BTree:

```
def __init__(self, order=4):  
    self.root = BTreeNode(True)  
    self.max_keys = order - 1  
  
def search(self, node, key):  
    i = 0  
  
    while i < len(node.keys) and key > node.keys[i]:  
        i += 1  
  
    if i < len(node.keys) and key == node.keys[i]:  
        return True  
  
    if node.leaf:  
        return False
```

```

    return self.search(node.children[i], key)

def split_child(self, parent, index):
    child = parent.children[index]
    new_node = BTreeNode(child.leaf)

    middle = len(child.keys) // 2
    middle_key = child.keys[middle]

    new_node.keys = child.keys[middle + 1:]
    child.keys = child.keys[:middle]

    if not child.leaf:
        new_node.children = child.children[middle + 1:]
        child.children = child.children[:middle + 1]

    parent.children.insert(index + 1, new_node)
    parent.keys.insert(index, middle_key)

def insert(self, key):
    root = self.root

    if len(root.keys) == self.max_keys:
        new_root = BTreeNode(False)
        new_root.children.append(root)
        self.root = new_root

        self.split_child(new_root, 0)
        self.insert_non_full(new_root, key)
    else:
        self.insert_non_full(root, key)

def insert_non_full(self, node, key):
    i = len(node.keys) - 1

```

```

if node.leaf:
    node.keys.append(key)
    node.keys.sort()
    return

while i >= 0 and key < node.keys[i]:
    i -= 1

i += 1

if len(node.children[i].keys) == self.max_keys:
    self.split_child(node, i)

    if key > node.keys[i]:
        i += 1

self.insert_non_full(node.children[i], key)

def display(self, node=None, level=0):
    if node is None:
        node = self.root

    print("Level", level, ":", node.keys)

    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

```

Test

```
btree = BTree(4)
```

```
keys = [10, 20, 5, 6, 12, 30, 7, 17]
```

```
for key in keys:
```

```
btree.insert(key)
```

```
print("B-Tree:")
```

```
btree.display()
```

```
print("\nSearch 12:", btree.search(btree.root, 12))
```

```
print("Search 25:", btree.search(btree.root, 25))
```

Expected Structure

Level 0 : [10, 20]

Level 1 : [5, 6, 7]

Level 1 : [12, 17]

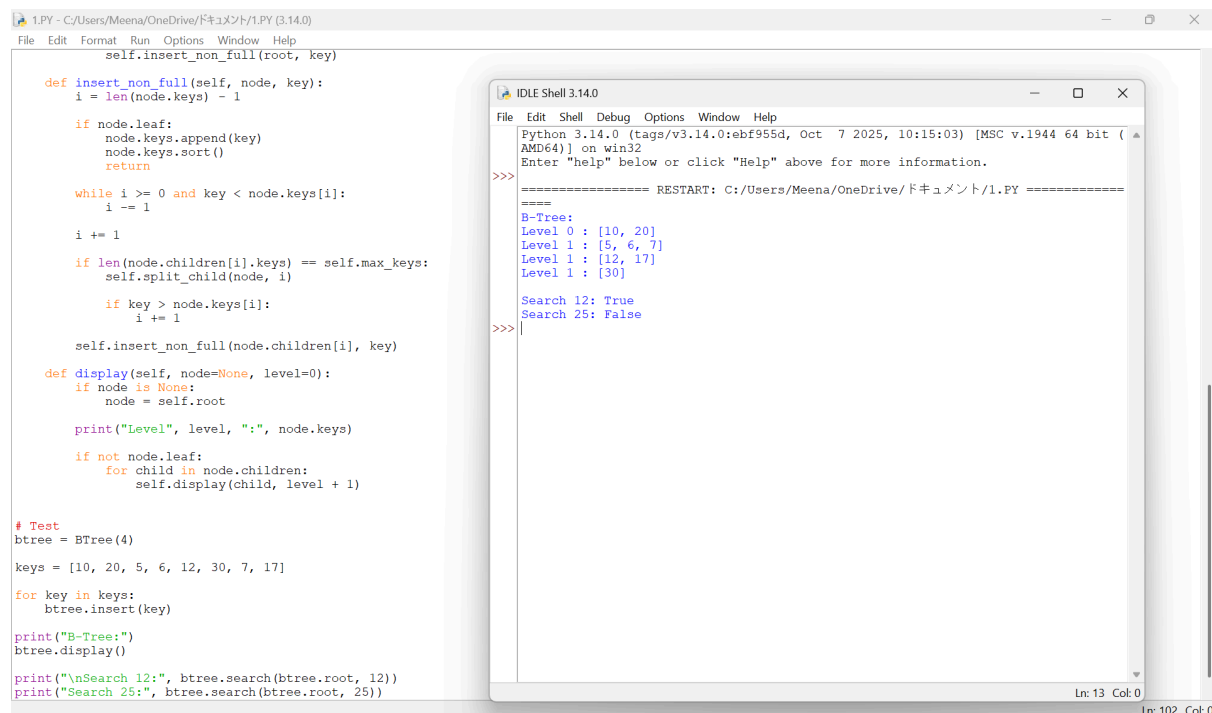
Level 1 : [30]

Complexity:

Insertion: $O(\log n)$

Search: $O(\log n)$

OUTPUT:



```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help
self.insert_non_full(root, key)

def insert_non_full(self, node, key):
    i = len(node.keys) - 1
    if node.leaf:
        node.keys.append(key)
        node.keys.sort()
        return
    while i >= 0 and key < node.keys[i]:
        i -= 1
    i += 1
    if len(node.children[i].keys) == self.max_keys:
        self.split_child(node, i)
        if key > node.keys[i]:
            i += 1
    self.insert_non_full(node.children[i], key)

def display(self, node=None, level=0):
    if node is None:
        node = self.root
    print("Level", level, ":", node.keys)
    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

# Test
btree = BTree(4)
keys = [10, 20, 5, 6, 12, 30, 7, 17]
for key in keys:
    btree.insert(key)
print("B-Tree:")
btree.display()

print("\nSearch 12:", btree.search(btree.root, 12))
print("Search 25:", btree.search(btree.root, 25))

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
>>>
B-Tree:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]
Search 12: True
Search 25: False
>>>
```


5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

The important feature is that leaf nodes are linked sequentially, making range queries efficient.

```
class BPlusNode:
```

```
    def __init__(self, leaf=False):
```

```
        self.leaf = leaf
```

```
        self.keys = []
```

```
        self.children = []
```

```
        self.next = None
```

```
class BPlusTree:
```

```
    def __init__(self, order=4):
```

```
        self.order = order
```

```
        self.root = BPlusNode(True)
```

```
    def find_leaf(self, key):
```

```
        node = self.root
```

```
        while not node.leaf:
```

```
            i = 0
```

```
            while i < len(node.keys) and key >= node.keys[i]:
```

```
                i += 1
```

```
            node = node.children[i]
```

```
        return node
```

```
    def insert(self, key):
```

```
        root = self.root
```

```

if root.leaf:
    root.keys.append(key)
    root.keys.sort()

    if len(root.keys) >= self.order:
        self.split_root()

    return

leaf = self.find_leaf(key)
leaf.keys.append(key)
leaf.keys.sort()

if len(leaf.keys) >= self.order:
    self.split_leaf(leaf)

def split_root(self):
    old_root = self.root

    left = BPlusNode(True)
    right = BPlusNode(True)

    mid = len(old_root.keys) // 2

    left.keys = old_root.keys[:mid]
    right.keys = old_root.keys[mid:]

    left.next = right

    new_root = BPlusNode(False)
    new_root.keys = [right.keys[0]]
    new_root.children = [left, right]

    self.root = new_root

```

```

def split_leaf(self, leaf):
    right = BPlusNode(True)

    mid = len(leaf.keys) // 2

    right.keys = leaf.keys[mid:]
    leaf.keys = leaf.keys[:mid]

    right.next = leaf.next
    leaf.next = right

def range_query(self, low, high):
    leaf = self.find_leaf(low)
    result = []

    while leaf:
        for key in leaf.keys:
            if low <= key <= high:
                result.append(key)

            if key > high:
                return result

        leaf = leaf.next

    return result

def display_leaves(self):
    leaf = self.root

    while not leaf.leaf:
        leaf = leaf.children[0]

    print("Leaf-level linked list:")

```

```
while leaf:
    print(leaf.keys, end=" -> ")
    leaf = leaf.next
```

```
print("None")
```

```
# Test
```

```
tree = BPlusTree()
```

```
for key in [10, 20, 5, 6, 12, 30, 7, 17, 25, 40]:
    tree.insert(key)
```

```
tree.display_leaves()
```

```
print("Range Query 10-30:")
print(tree.range_query(10, 30))
```

Important advantage:

Leaf 1 -> Leaf 2 -> Leaf 3 -> Leaf 4

The linked leaves allow range queries without repeatedly traversing from the root.

OUTPUT:

```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help

class BPlusNode:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None

class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = BPlusNode(True)

    def find_leaf(self, key):
        node = self.root

        while not node.leaf:
            i = 0

            while i < len(node.keys) and key >= node.keys[i]:
                i += 1

            node = node.children[i]

        return node

    def insert(self, key):
        root = self.root

        if root.leaf:
            root.keys.append(key)
            root.keys.sort()

            if len(root.keys) >= self.order:
                self.split_root()

            return

        leaf = self.find_leaf(key)
        leaf.keys.append(key)
        leaf.keys.sort()

        if len(leaf.keys) >= self.order:
            self.split_leaf(leaf)

    def split_root(self):
        old root = self.root
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
>>>
Leaf-level linked list:
[5, 6, 7] -> [10, 12, 40] -> [17, 25] -> [20, 30] -> None
Range Query 10-30:
[10, 12]

>>>
```

Ln: 9 Col: 0

6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

A dense index contains an index entry for every record.

```
class DenseIndex:
    def __init__(self, records):
        self.records = sorted(records)
        self.index = [(key, position)
                      for position, (key, name) in enumerate(self.records)]

    def search(self, key):
        low = 0
        high = len(self.index) - 1

        while low <= high:
            mid = (low + high) // 2

            if self.index[mid][0] == key:
                position = self.index[mid][1]
                return self.records[position]

            elif self.index[mid][0] < key:
                low = mid + 1
            else:
                high = mid - 1

        return None

    def display_index(self):
        print("Dense Index")
        print("Key\tPointer")

        for key, pointer in self.index:
```

```
print(key, "\t", pointer)
```

```
# Test
```

```
records = [  
    (101, "Arun"),  
    (102, "Bala"),  
    (103, "Charan"),  
    (104, "Divya"),  
    (105, "Ezhil")  
]
```

```
dense = DenseIndex(records)
```

```
dense.display_index()
```

```
print("\nSearch 103:")
```

```
print(dense.search(103))
```

Dense Index

Key	Pointer
101	0
102	1
103	2
104	3
105	4

Search complexity: $O(\log n)$

OUTPUT:

```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help

class DenseIndex:
    def __init__(self, records):
        self.records = sorted(records)
        self.index = [(key, position)
                       for position, (key, name) in enumerate(self.records)]

    def search(self, key):
        low = 0
        high = len(self.index) - 1
        while low <= high:
            mid = (low + high) // 2
            if self.index[mid][0] == key:
                position = self.index[mid][1]
                return self.records[position]
            elif self.index[mid][0] < key:
                low = mid + 1
            else:
                high = mid - 1
        return None

    def display_index(self):
        print("Dense Index")
        print("Key\tPointer")
        for key, pointer in self.index:
            print(key, "\t", pointer)

# Test
records = [
    (101, "Arun"),
    (102, "Bala"),
    (103, "Charan"),
    (104, "Divya"),
    (105, "Ezhil")
]

dense = DenseIndex(records)

dense.display_index()

print("\nSearch 103:")
print(dense.search(103))
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
Dense Index
Key      Pointer
101      0
102      1
103      2
104      3
105      4

Search 103:
(103, 'Charan')
>>>
```


7. Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

This program demonstrates directory doubling when bucket overflow occurs.

```
BUCKET_SIZE = 2
```

```
class Bucket:
```

```
    def __init__(self, depth):
        self.depth = depth
        self.records = []
```

```
class ExtendibleHash:
```

```
    def __init__(self):
        self.global_depth = 1
```

```
        self.directory = [
            Bucket(1),
            Bucket(1)
        ]
```

```
    def hash_value(self, key):
        return key & ((1 << self.global_depth) - 1)
```

```
    def insert(self, key):
        index = self.hash_value(key)
        bucket = self.directory[index]
```

```
        if key in bucket.records:
            return
```

```
        if len(bucket.records) < BUCKET_SIZE:
```

```
    bucket.records.append(key)
    return
```

```
self.split_bucket(index)
self.insert(key)
```

```
def split_bucket(self, index):
    old_bucket = self.directory[index]

    if old_bucket.depth == self.global_depth:
        old_directory = self.directory[:]

        self.global_depth += 1

        self.directory = old_directory + old_directory[:]

        print("\nDirectory doubled.")
        print("Global Depth =", self.global_depth)

    new_depth = old_bucket.depth + 1

    new_bucket = Bucket(new_depth)
    old_bucket.depth = new_depth

    pattern = index & ((1 << (new_depth - 1)) - 1)

    for i in range(len(self.directory)):
        if self.directory[i] is old_bucket:
            if ((i >> (new_depth - 1)) & 1) == 1:
                self.directory[i] = new_bucket

    old_records = old_bucket.records[:]
    old_bucket.records = []
```

```

for record in old_records:
    new_index = self.hash_value(record)
    self.directory[new_index].records.append(record)

```

```

def display(self):
    print("\nDirectory:")
    for i, bucket in enumerate(self.directory):
        print(
            format(i, f"0{self.global_depth}b"),
            "->",
            bucket.records,
            "Local Depth:",
            bucket.depth
        )

```

Insert 8 records

```
eh = ExtendibleHash()
```

```
for key in [1, 2, 3, 4, 5, 6, 7, 8]:
```

```
    print("\nInsert:", key)
```

```
    eh.insert(key)
```

```
    eh.display()
```

Expected Concept

Initial Directory

0 -> Bucket

1 -> Bucket

↓ Overflow

Directory Doubles

00

01

10

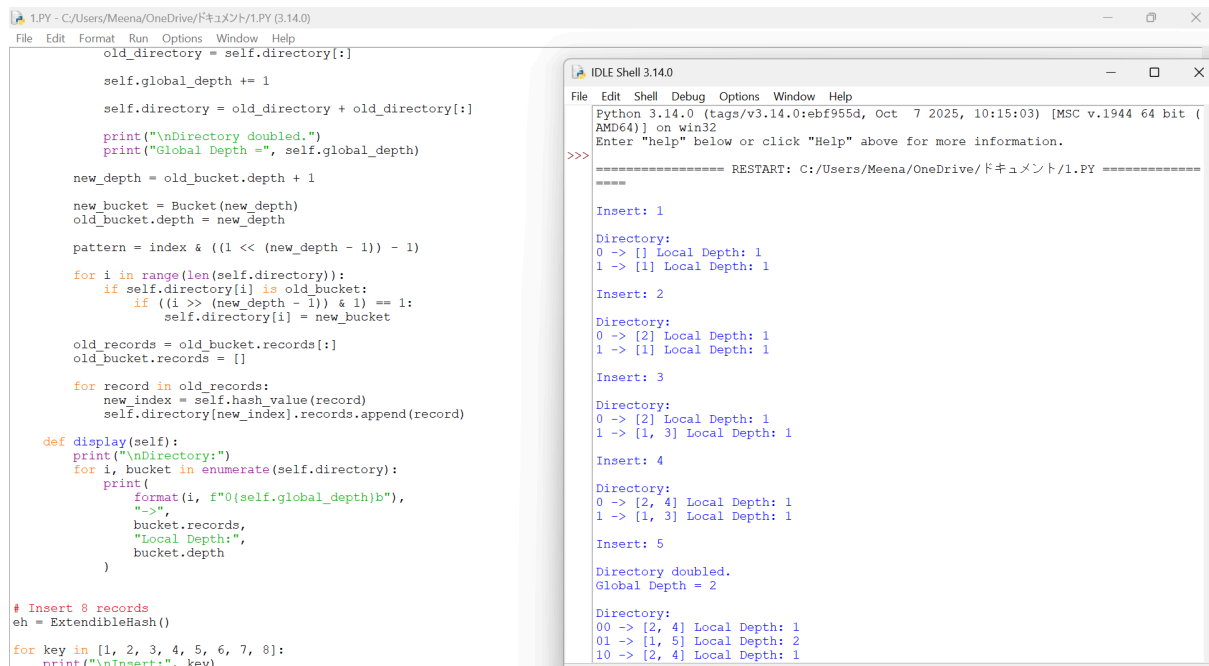
11

The directory grows dynamically rather than using a fixed hash table size.

Complexity:

- Average insertion: $O(1)$
- Average search: $O(1)$

OUTPUT:



```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help

self.global_depth += 1
self.directory = old_directory + old_directory[:]
print("\nDirectory doubled.")
print("Global Depth =", self.global_depth)

new_depth = old_bucket.depth + 1
new_bucket = Bucket(new_depth)
old_bucket.depth = new_depth

pattern = index & ((1 << (new_depth - 1)) - 1)
for i in range(len(self.directory)):
    if self.directory[i] is old_bucket:
        if ((i >> (new_depth - 1)) & 1) == 1:
            self.directory[i] = new_bucket

old_records = old_bucket.records[:]
old_bucket.records = []

for record in old_records:
    new_index = self.hash_value(record)
    self.directory[new_index].records.append(record)

def display(self):
    print("\nDirectory:")
    for i, bucket in enumerate(self.directory):
        print(
            format(i, f"0{self.global_depth}b"),
            "->",
            bucket.records,
            "Local Depth:",
            bucket.depth
        )

# Insert 8 records
eh = ExtendibleHash()
for key in [1, 2, 3, 4, 5, 6, 7, 8]:
    print("\nInsert:", key)
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
>>>

Insert: 1
Directory:
0 -> [] Local Depth: 1
1 -> [1] Local Depth: 1

Insert: 2
Directory:
0 -> [2] Local Depth: 1
1 -> [1] Local Depth: 1

Insert: 3
Directory:
0 -> [2] Local Depth: 1
1 -> [1, 3] Local Depth: 1

Insert: 4
Directory:
0 -> [2, 4] Local Depth: 1
1 -> [1, 3] Local Depth: 1

Insert: 5
Directory doubled.
Global Depth = 2
Directory:
00 -> [2, 4] Local Depth: 1
01 -> [1, 5] Local Depth: 2
10 -> [2, 4] Local Depth: 1
```

```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help
old_directory = self.directory[:]

self.global_depth += 1

self.directory = old_directory + old_directory[:]

print("\nDirectory doubled.")
print("Global Depth =", self.global_depth)

new_depth = old_bucket.depth + 1

new_bucket = Bucket(new_depth)
old_bucket.depth = new_depth

pattern = index & ((1 << (new_depth - 1)) - 1)

for i in range(len(self.directory)):
    if self.directory[i] is old_bucket:
        if ((i >> (new_depth - 1)) & 1) == 1:
            self.directory[i] = new_bucket

old_records = old_bucket.records[:]
old_bucket.records = []

for record in old_records:
    new_index = self.hash_value(record)
    self.directory[new_index].records.append(record)

def display(self):
    print("\nDirectory:")
    for i, bucket in enumerate(self.directory):
        print(
            format(i, f"0{self.global_depth}b"),
            "->",
            bucket.records,
            "Local Depth:",
            bucket.depth
        )

# Insert 8 records
eh = ExtendibleHash()

for key in [1, 2, 3, 4, 5, 6, 7, 8]:
    print("\nInsert:", key)
    eh.insert(key)
    eh.display()
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

Directory:
0 -> [2, 4] Local Depth: 1
1 -> [1, 3] Local Depth: 1

Insert: 5

Directory doubled.
Global Depth = 2

Directory:
00 -> [2, 4] Local Depth: 1
01 -> [1, 5] Local Depth: 2
10 -> [2, 4] Local Depth: 1
11 -> [3] Local Depth: 2

Insert: 6

Directory:
00 -> [4] Local Depth: 2
01 -> [1, 5] Local Depth: 2
10 -> [2, 6] Local Depth: 2
11 -> [3] Local Depth: 2

Insert: 7

Directory:
00 -> [4] Local Depth: 2
01 -> [1, 5] Local Depth: 2
10 -> [2, 6] Local Depth: 2
11 -> [3, 7] Local Depth: 2

Insert: 8

Directory:
00 -> [4, 8] Local Depth: 2
01 -> [1, 5] Local Depth: 2
10 -> [2, 6] Local Depth: 2
11 -> [3, 7] Local Depth: 2

>>>
```

Ln: 64 Col: C

Ln: 87 Col: 0

29°C
Partly cloudy

Search

21:11
26-08-2026

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Assume:

1000 records

10 records per block

Therefore:

Number of blocks = $1000 / 10 = 100$

import math

```
def calculate_io(records, records_per_block, retrieval_count):
```

```
    blocks = math.ceil(records / records_per_block)
```

```
    sequential_io = blocks
```

```
    indexed_io = 1 + math.ceil(math.log2(blocks))
```

```
    print("Total Records:", records)
```

```
    print("Records per Block:", records_per_block)
```

```
    print("Total Blocks:", blocks)
```

```
    print("\nSequential Retrieval I/O:", sequential_io)
```

```
    print("Indexed Retrieval I/O:", indexed_io)
```

```
    print("\nI/O Reduction:",  
          sequential_io - indexed_io)
```

```
calculate_io(1000, 10, 1)
```

Expected Result

Total Records: 1000

Records per Block: 10

Total Blocks: 100

Sequential Retrieval I/O: 100

Indexed Retrieval I/O: 8

I/O Reduction: 92

Comparison

Method	I/O Operations
Sequential	100
Indexed	8

Thus, indexed retrieval significantly reduces secondary-storage I/O.

9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

```
import time
import bisect
```

```
N = 100000
```

```
# Sorted file
```

```
records = [(i, "Student" + str(i)) for i in range(N)]
```

```
# ----- DENSE INDEX -----
```

```
dense_index = [(key, position)
                for position, (key, name) in enumerate(records)]
```

```
dense_keys = [x[0] for x in dense_index]
```

```
# ----- SPARSE INDEX -----
```

```
BLOCK_SIZE = 100
```

```
sparse_index = []
```

```
for position in range(0, N, BLOCK_SIZE):
    sparse_index.append((records[position][0], position))
```

```
sparse_keys = [x[0] for x in sparse_index]
```

```
def dense_search(key):
```



```
pos = bisect.bisect_left(dense_keys, key)
```

```
if pos < len(dense_keys) and dense_keys[pos] == key:  
    return dense_index[pos][1]
```

```
return -1
```

```
def sparse_search(key):
```

```
    pos = bisect.bisect_right(sparse_keys, key) - 1
```

```
    if pos < 0:
```

```
        pos = 0
```

```
    start = sparse_index[pos][1]
```

```
    end = min(start + BLOCK_SIZE, N)
```

```
    for i in range(start, end):
```

```
        if records[i][0] == key:
```

```
            return i
```

```
    return -1
```

```
# Timing
```

```
search_key = 98765
```

```
start = time.perf_counter()
```

```
for _ in range(1000):
```

```
    dense_search(search_key)
```

```
dense_time = time.perf_counter() - start
```

```
start = time.perf_counter()

for _ in range(1000):
    sparse_search(search_key)

sparse_time = time.perf_counter() - start

print("Dense Index Time :", dense_time)
print("Sparse Index Time:", sparse_time)

print("\nDense Search Result:", records[dense_search(search_key)])
print("Sparse Search Result:", records[sparse_search(search_key)])
```

Complexity

Dense index:

Search = $O(\log n)$

Index size = $O(n)$

Sparse index:

Index size = $O(n / \text{block_size})$

Search = $O(\log(n/\text{block_size}) + \text{block_size})$

The timing values depend on the computer, so the program measures them dynamically rather than using a fixed value.

OUTPUT:

```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help

import time
import bisect

N = 100000

# Sorted file
records = [(i, "Student" + str(i)) for i in range(N)]

# ----- DENSE INDEX -----
dense_index = [(key, position)
               for position, (key, name) in enumerate(records)]
dense_keys = [x[0] for x in dense_index]

# ----- SPARSE INDEX -----
BLOCK_SIZE = 100
sparse_index = []
for position in range(0, N, BLOCK_SIZE):
    sparse_index.append((records[position][0], position))
sparse_keys = [x[0] for x in sparse_index]

def dense_search(key):
    pos = bisect.bisect_left(dense_keys, key)
    if pos < len(dense_keys) and dense_keys[pos] == key:
        return dense_index[pos][1]
    return -1

def sparse_search(key):
    pos = bisect.bisect_right(sparse_keys, key) - 1
    if pos < 0:
        pos = 0
    start = sparse_index[pos][1]
    end = min(start + BLOCK_SIZE, N)
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
Dense Index Time : 0.000365999929368496
Sparse Index Time: 0.004355099983513355
Dense Search Result: (98765, 'Student98765')
Sparse Search Result: (98765, 'Student98765')
>>>
```

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

This program constructs a B+ tree, traverses it, displays the linked leaves, and performs the required range query.

```
class Node:
```

```
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None
```

```
class BPlusTree:
```

```
    def __init__(self, order=4):
        self.order = order
        self.root = Node(True)
```

```
    def find_leaf(self, key):
        current = self.root
```

```
        while not current.leaf:
            i = 0
```

```
            while i < len(current.keys) and key >= current.keys[i]:
                i += 1
```

```
            current = current.children[i]
```

```
        return current
```

```
    def insert(self, key):
```

```
        leaf = self.find_leaf(key)
```

```

leaf.keys.append(key)
leaf.keys.sort()

if len(leaf.keys) >= self.order:
    self.split_leaf(leaf)

def split_leaf(self, leaf):
    right = Node(True)

    middle = len(leaf.keys) // 2

    right.keys = leaf.keys[middle:]
    leaf.keys = leaf.keys[:middle]

    right.next = leaf.next
    leaf.next = right

    # Simplified root handling for demonstration
    if leaf == self.root:
        new_root = Node(False)

        new_root.keys = [right.keys[0]]
        new_root.children = [leaf, right]

        self.root = new_root

def traverse(self, node=None, level=0):
    if node is None:
        node = self.root

    print("Level", level, ":", node.keys)

    if not node.leaf:
        for child in node.children:

```

```
        self.traverse(child, level + 1)
```

```
def get_first_leaf(self):
```

```
    node = self.root
```

```
    while not node.leaf:
```

```
        node = node.children[0]
```

```
    return node
```

```
def range_query(self, low, high):
```

```
    result = []
```

```
    leaf = self.find_leaf(low)
```

```
    while leaf:
```

```
        for key in leaf.keys:
```

```
            if key > high:
```

```
                return result
```

```
            if key >= low:
```

```
                result.append(key)
```

```
        leaf = leaf.next
```

```
    return result
```

```
def display_leaves(self):
```

```
    leaf = self.get_first_leaf()
```

```
    print("\nLeaf Level:")
```

```

while leaf:
    print(leaf.keys, end=" -> ")
    leaf = leaf.next

print("None")

# Create B+ Tree
bpt = BPlusTree(4)

keys = [
    5, 10, 15, 20, 25,
    30, 35, 40, 45, 50,
    55, 60
]

for key in keys:
    bpt.insert(key)

print("B+ Tree Traversal:")
bpt.traverse()

bpt.display_leaves()

print("\nRange Query: 15 to 45")

result = bpt.range_query(15, 45)

print("Matching Records:", result)

```

Expected Range Query Output

Range Query: 15 to 45

Matching Records:

[15, 20, 25, 30, 35, 40, 45]

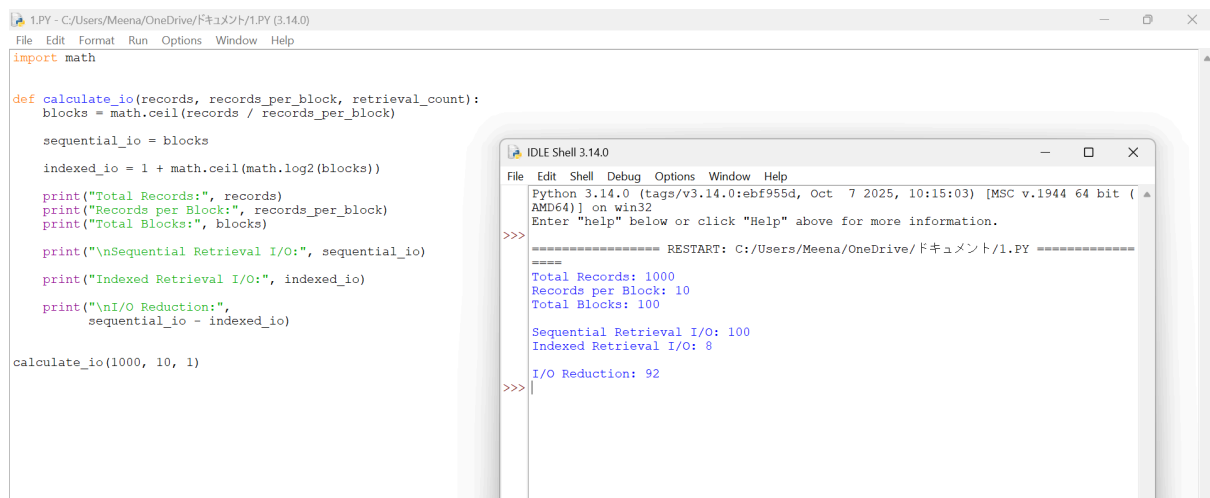
Why B+ Tree is efficient

The search first reaches the appropriate leaf and then follows the leaf-level linked list.

15 → 20 → 25 → 30 → 35 → 40 → 45

This makes range retrieval efficient.

OUTPUT:



The image shows a screenshot of a Python IDE with two windows. The main window displays a Python script for calculating I/O for sequential and indexed retrieval. The script defines a function `calculate_io` that takes `records`, `records_per_block`, and `retrieval_count` as arguments. It calculates the number of blocks and then the sequential and indexed I/O. The script is executed with `calculate_io(1000, 10, 1)`. A smaller window titled 'IDLE Shell 3.14.0' shows the output of the script.

```
1.PY - C:/Users/Meena/OneDrive/ドキュメント/1.PY (3.14.0)
File Edit Format Run Options Window Help
import math

def calculate_io(records, records_per_block, retrieval_count):
    blocks = math.ceil(records / records_per_block)

    sequential_io = blocks

    indexed_io = 1 + math.ceil(math.log2(blocks))

    print("Total Records:", records)
    print("Records per Block:", records_per_block)
    print("Total Blocks:", blocks)

    print("\nSequential Retrieval I/O:", sequential_io)

    print("Indexed Retrieval I/O:", indexed_io)

    print("\nI/O Reduction:",
          sequential_io - indexed_io)

calculate_io(1000, 10, 1)
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Meena/OneDrive/ドキュメント/1.PY =====
Total Records: 1000
Records per Block: 10
Total Blocks: 100
Sequential Retrieval I/O: 100
Indexed Retrieval I/O: 8
I/O Reduction: 92
>>>
```